

Multi-Variable Control Systems Project-Update 4

Alek Krupka

May 2026

1 Introduction

In this project update, system identification methods will be used to perform a frequency analysis on data from a system, fit the data to a model, compare the identified model to the real model. Noise is then added to the model, and the system is re-evaluated to see noise's effect on the identified system. Finally, a robust controller will be designed that includes the model identification error.

2 Plant Model

The plant model was implemented using the equations derived in previous project updates but this time through a Simulink model. Using a Simulink model is useful for the following two reasons.

1. It allows for easily adjusting the parameters of the plant through the plant mask as shown by Figure 2.
2. It allows for easily performing a frequency analysis to create a frd model.

First, the modelled plant is shown below by

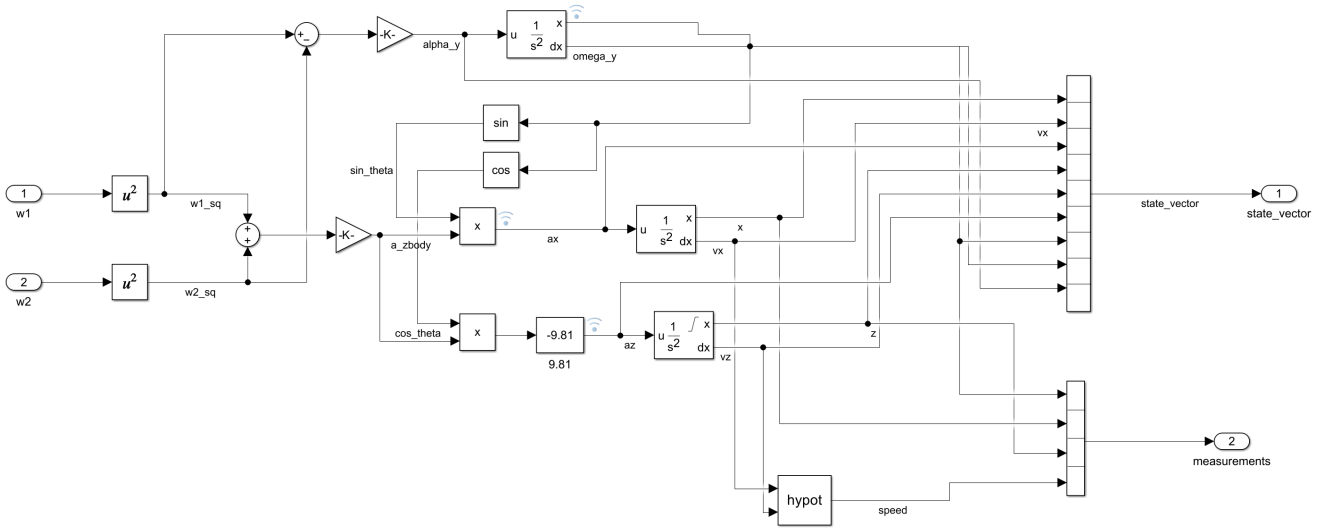


Figure 1: Drone Simulink Model

And the open loop system containing the plant is given by Figure 2. As observed, the parameters are given to the right and can easily be adjusted.

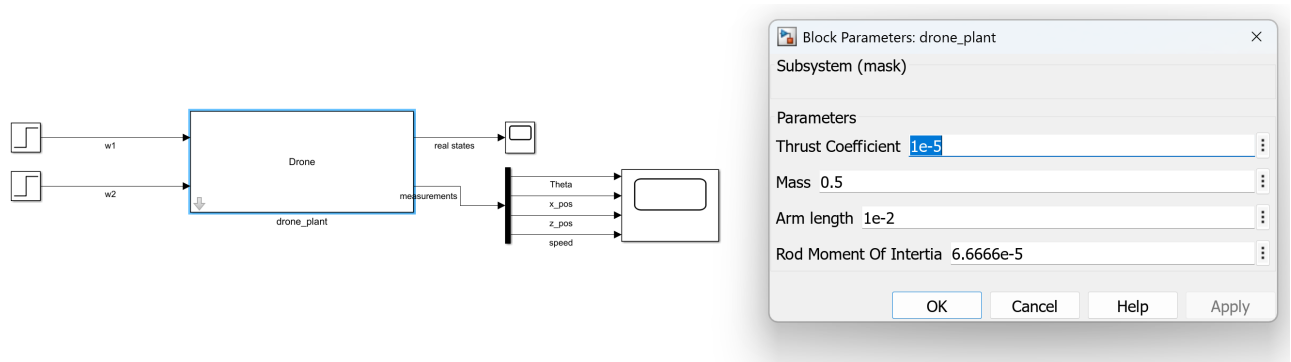


Figure 2: High Level Simulink Model with Parameter Options

A lot of the work can be done through Simulink's GUI which will be useful for the upcoming analysis.

3 Finding the Frequency Response Data

Using the model, a created a chirp signal can now be given to the plant to create data for the system which will then be used for system identification. Simulink has a very convenient way of collecting this data through the "frequency response estimator" app. Performing a frequency analysis using the chirp signal can be done through the following steps in Simulink.

1. Go to the ESTIMATOR TAB.
2. Select "Input Signal" in the top.
3. Select "Chirp" from the dropdown

Selecting "Chirp" allows for the frequency response to be estimated using the chirp frequency. Note that other options can be used. However, the chirp input was selected since it covers a wide range and frequencies and since it is a simulation, we don't really care about the long time it takes to analyze using this method.

The chirp signal additionally allows for the model to use different parameters for the frequency sweep as shown in Figure 3

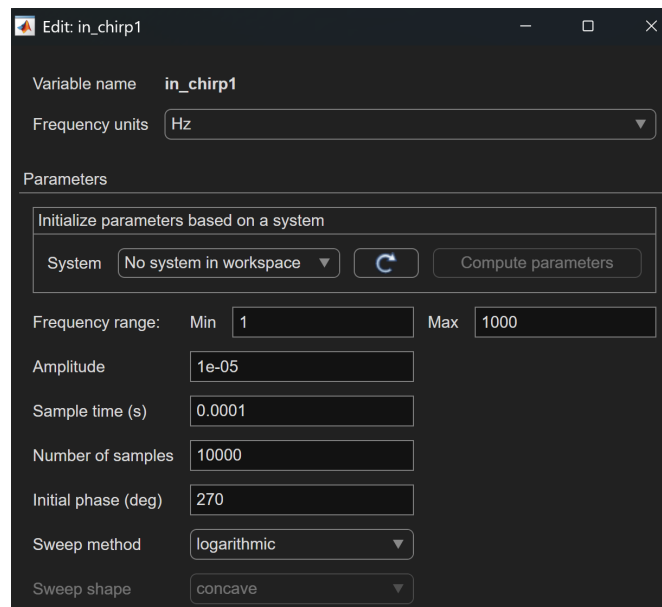


Figure 3: Frequency Sweep Selected Parameters

As shown, the frequency is swept between 1Hz and 1kHz. While higher frequencies could be analyzed, it is more realistic to keep the sampling time relatively low since the sensors are most likely very slow. In reality, when performing a system identification on a model like this, the frequency would be restricted by sensor speed and interrupt time within the microcontroller.

Next, Simulink will automatically generate the frd model from the data collected through the chirp input. This frd model can then be exported to the MATLAB workspace where further analysis can be performed. Note that for the MATLAB live script to function, the "chirp_frd" variable must be exported to the MATLAB workspace. The frd model is essentially data taken at a variety of frequencies and shows the channels between inputs and outputs. Once this is done though, the frd model can be converted to a state space using the MATLAB system identification toolbox.

4 System Estimation

Once the frd model is located in the MATLAB workspace there are two options to obtain the estimated state space of the system.

1. Use the built-in MATLAB function ssest.
2. Use the built-in MATLAB function n4sid.

Note that the fitfrd function cannot be used here due to the frd being one of a MIMO control system.

4.1 Model Order Estimation

To estimate the model order using these functions, the following approach can be used as specified in [1]. Run the ssest/n4sid function where the second parameter is an array from 1-n where n is the max order to test. In my case, I tested 200 states. States with small Henkel singular values are eliminated, and the final model order only contains the relatively large singular values. However, MATLAB opens a pop-up window and just recommended a model order to use as shown by Figure 4.

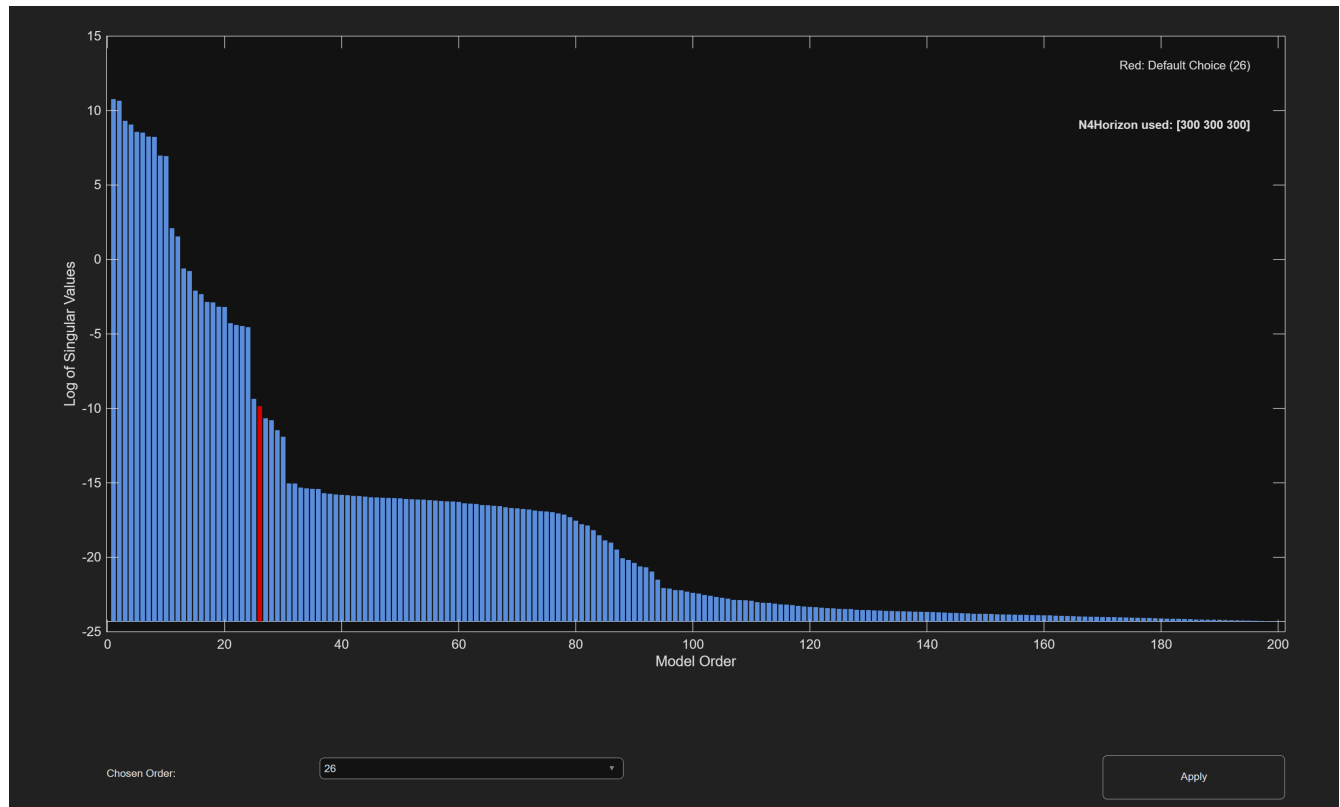


Figure 4: Model Order Estimation using ssest

As shown, the recommended model order is 26 states. One problem though is that this might be too high of an order and will cause too much noise in the model. However, the exact model order will be decided at a later time when developing the controller

4.2 The Estimated State Space

Now that the command is finally run and the model order is estimated the state space is generated. Note that because the model is of relatively high order, the exact space needs to be observed from the MATLAB live script attached in the appendix.

4.3 Comparing the Estimated System to the True System

While comparing the estimated to the actual system is almost impossible, the bode plots of both MIMO systems can easily be compared.

The original system's state space can be generated through Simulink's Linear Analysis tool. The tool linearizes around an operating point and automatically generates the bode plots. Note that the operating point selected is the initial conditions of the plant where there is no velocity and the position is above ground. This also calculates the bode plot for every channel of the MIMO system as shown by Figure 5.

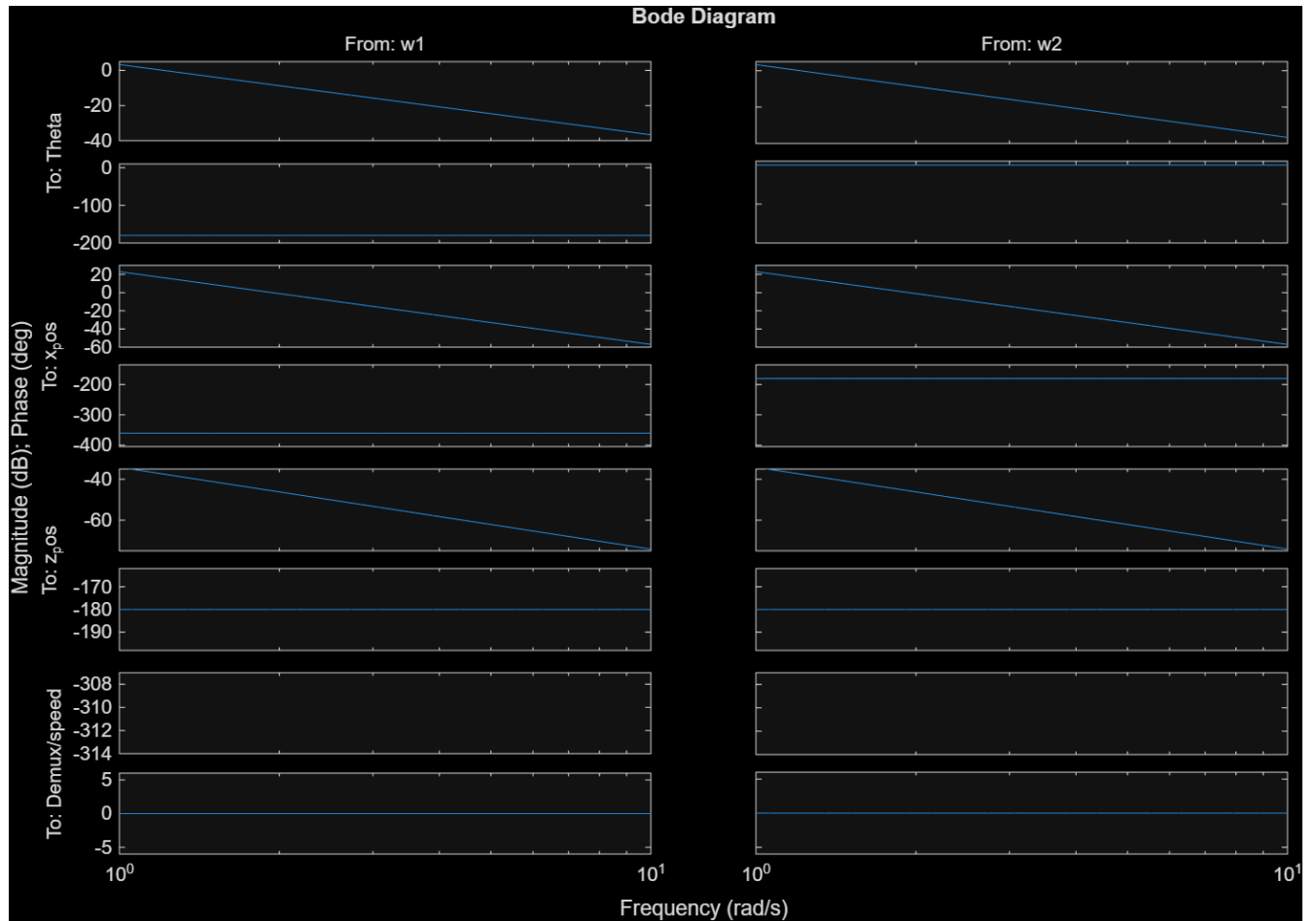


Figure 5: Bode Plot for the Linearized Open Loop Drone Plant

The bode plot can again be generated for the identified state space model as shown by Figure 6.

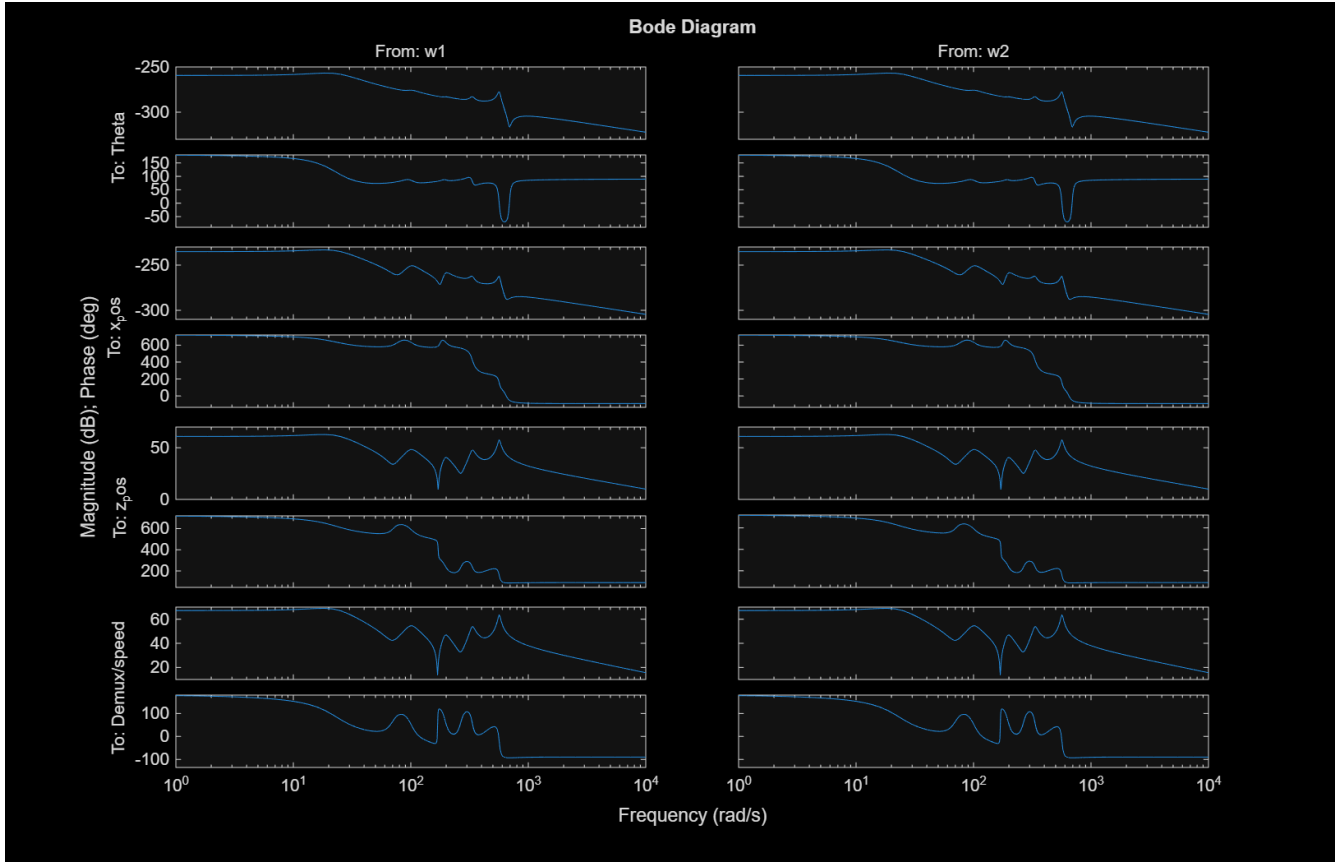


Figure 6: Bode Plot for Identified State Space Model

As can be observed, the bode plots from the linearized model (Figure 5) look very different from the bode plots generated by the identified model (Figure 6). The result is not surprising though, as the default operating point for the drone is upright, meaning the linearization is not super accurate in describing the state space model. When identifying the system, the drone deviates from the assumed operating point causing the bode plots to look more natural.

5 Simulating Noise on the Output

Now, we add measurement noise to the output of the system when taking the frd response. To do this, white noise was added to each sensor output. How the Simulink model was changed to accommodate for adding noise is given by Figure 7.

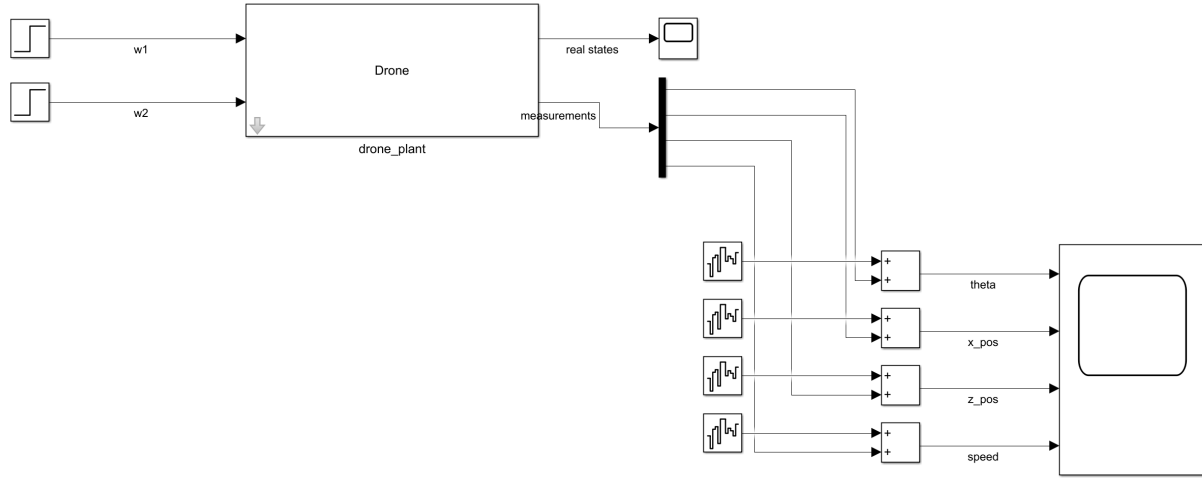


Figure 7: Simulink Model with White Noise

The same steps as before we repeated to find the frd of the system with output noise. The spectral densities for each band of white noise is given by the table below.

Sensor	Spectral Density
θ	0.1
x_{pos}	3
z_{pos}	5
speed	2

Performing the same steps as below the frd produces the bode plot below.

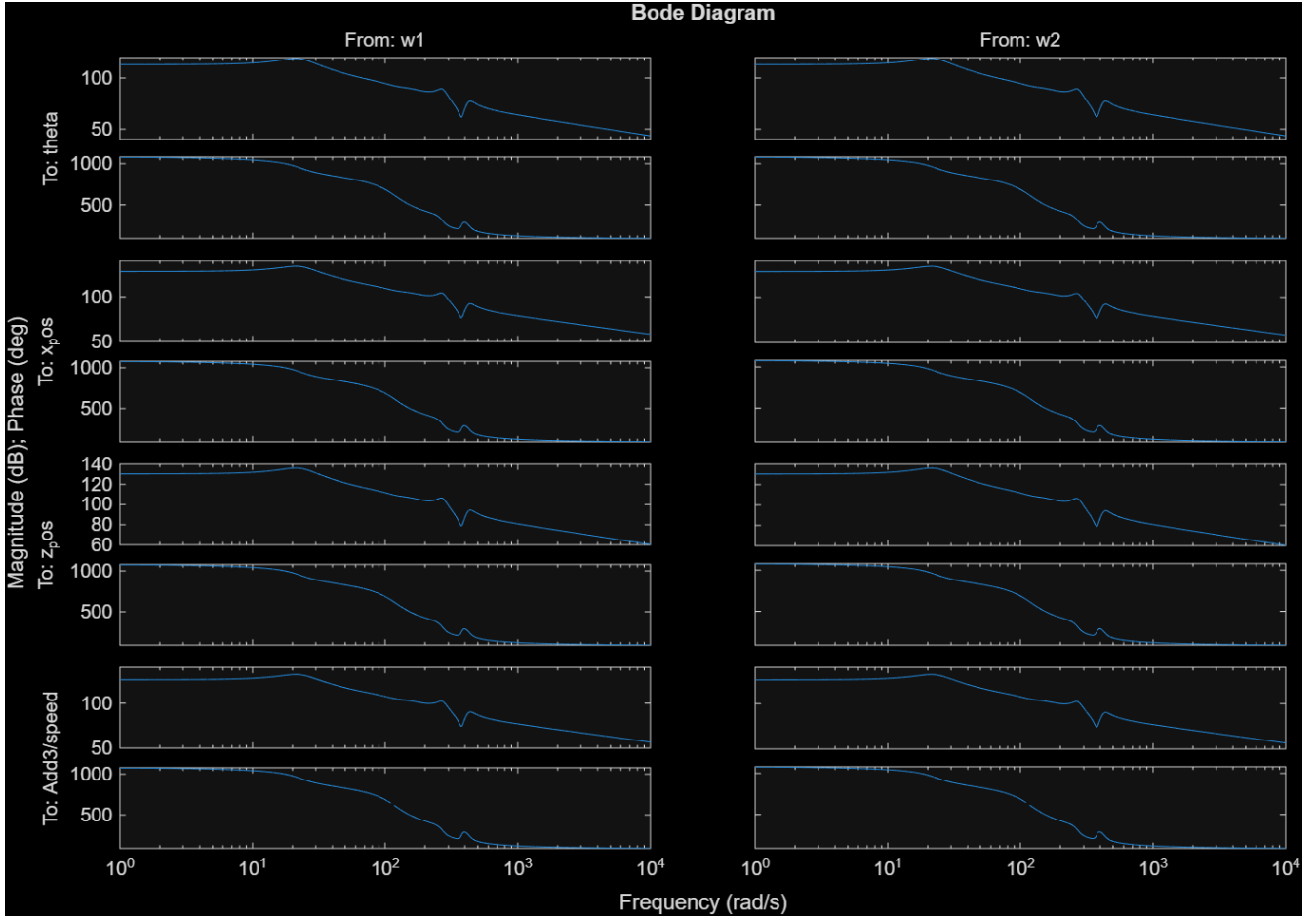


Figure 8: Noisy Model Bode Plots

As observed, the bode plots are fairly similar except the phase margin is far worse than desired. This again makes sense though because the system is not inherently stable and is incredibly non-linear.

Next, the chirp signal is run on the estimated state space to generate a new frd. This will be used in the code to design a robust controller.

6 Designing a Robust Controller using the frd

Next, a robust controller is designed to stabilize the system, closing the loop and allowing it to be controller by a reference. Using the frequency response data the problem can be formulated as an H_∞ problem with uncertainty where the uncertainty is the frequency response error between the new frd and the originally derived frd. See the MATLAB script on GITHUB for the final controller design and code for developing the robust controller.

7 Sources

1. <https://www.mathworks.com/help/ident/ref/n4sid.html>

8 Github Repo

<https://github.com/NightHydra/Multivariable-Control-Systems-Drone-Controller-Project>